UNIVERSIDAD POLITÉCNICA BICENTENARIO

Sistema automatizado de control de nivel de agua mediante ESP32

Presenta:

- **Ailton Alejandro Trujillo Lopez. 24033482**
- **Edgar Ivan Gutierrez Gutierrez. 24033101**
- **Ruben Antonio Arredondo Villegas. 24032957**
- **Roberto Leonel Mendoza Nuñez. 24033370**

Asignatura:

SISTEMAS EMBEBIDOS

Profesor:

Hugo Antonio Méndez Guzmán

Silao, Gto. a 15 de octubre de 2026

Contenido

Introducción 3

Marco Teórico 3

Desarrollo 4

Resultados 12

Observaciones y Conclusiones 13

Referencias 13

Lista de Figuras

Figura 1. Circuito electrónico del sistema de control de nivel de agua 5

Figura 2. Montaje final del sistema de control de nivel de agua 12

Introducción

En la presente práctica se desarrolló un sistema automatizado para el monitoreo y control del nivel de agua en un recipiente, mediante el uso de un microcontrolador ESP32 programado en el entorno de Arduino. El propósito principal del proyecto es mantener el nivel del agua dentro de un rango establecido, controlando de manera automática los procesos de llenado y vaciado del recipiente.

Para llevar a cabo este funcionamiento, se implementó un sensor ultrasónico encargado de medir la distancia entre el sensor y la superficie del agua, lo que permite calcular el porcentaje de llenado y el volumen aproximado de líquido contenido. Asimismo, se utilizaron dos bombas controladas mediante un módulo de relevadores: una para introducir agua al recipiente y otra para extraerla. El sistema cuenta también con una pantalla LCD para visualizar el nivel actual, el volumen de agua y los límites máximos y mínimos configurados.

Además, se incorporaron botones físicos que permiten modificar los parámetros de funcionamiento, así como un sistema de almacenamiento de datos para conservar la configuración establecida incluso después de interrumpir la alimentación eléctrica. También se programaron mecanismos de seguridad para detener las bombas cuando se detectan fallas consecutivas en las lecturas del sensor.

Mediante esta práctica, se busca aplicar conocimientos de programación, electrónica y automatización, integrando diferentes componentes para desarrollar un sistema de control capaz de operar de forma continua, precisa y segura.

Marco Teórico

El control automático del nivel de agua es un proceso que permite mantener una cantidad determinada de líquido dentro de un recipiente sin tener que estar supervisándolo manualmente todo el tiempo. Para lograrlo, se utilizan diferentes componentes electrónicos que trabajan en conjunto para medir el nivel del agua y controlar su entrada y salida.

En esta práctica se utiliza un ESP32, que es el encargado de controlar el funcionamiento de todo el sistema. También se emplea un sensor ultrasónico que mide la distancia entre el sensor y la superficie del agua, permitiendo conocer aproximadamente qué tan lleno está el recipiente.

Para controlar el llenado y el vaciado se utilizan dos bombas de agua conectadas mediante relevadores. El sistema activa una bomba para llenar el recipiente y otra para extraer el agua, dependiendo de los niveles máximo y mínimo establecidos en el programa.

Además, se incorpora una pantalla LCD que muestra el porcentaje de agua y su volumen aproximado en litros. También cuenta con botones que permiten modificar los niveles de llenado y una memoria que guarda la configuración para conservarla cuando el sistema se apaga.

En general, esta práctica permite comprender cómo se pueden combinar la programación y los componentes electrónicos para crear un sistema automatizado que facilite el control del nivel de agua y reduzca la necesidad de intervención manual.

Desarrollo

Material requerido

- 1 tarjeta ESP32.
- 1 sensor ultrasónico HC-SR04.
- 2 bombas de agua.
- 1 módulo de relevadores de 2 canales.
- 1 pantalla LCD de 16x2 con módulo I2C.
- 3 botones pulsadores.
- Cables Dupont.
- Protoboard.
- Cable USB para conectar y programar el ESP32.
- Computadora con Arduino IDE.
- Alimentación eléctrica adecuada para las bombas.

Como primera parte se desarrolló …

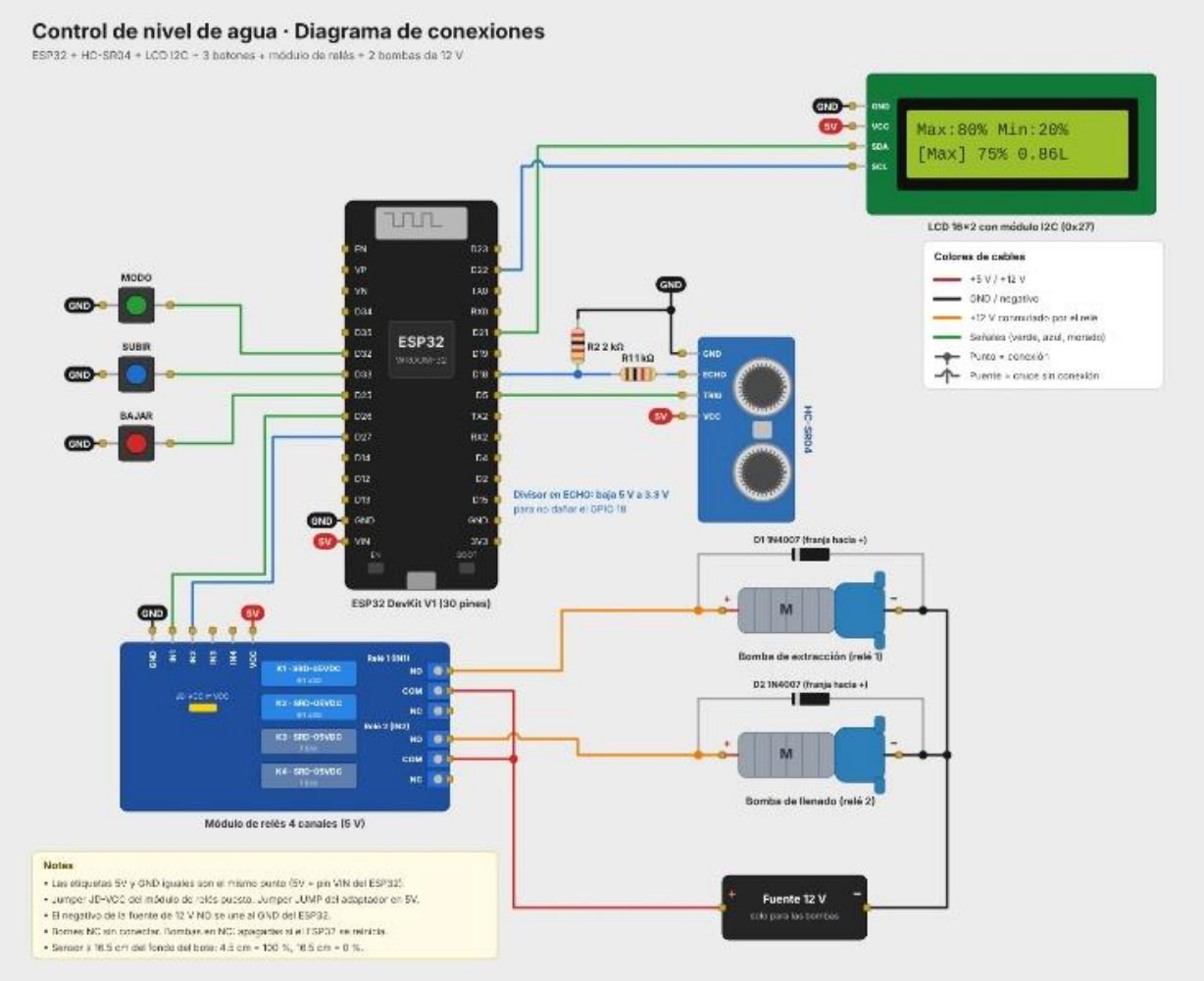


Figura 1. Circuito electrónico del sistema de control de nivel de agua

Enseguida se procedió a …

A continuación, se muestra el código empleado.

Código utilizado en el IDLE de Arduino
/* Botones: - MODO -> cada pulsacion cambia entre editar el MAX y editar el MIN. - SUBIR -> sube 1% el valor del modo seleccionado. - BAJAR -> baja 1% el valor del modo seleccionado. Los valores se guardan en memoria y se conservan aunque se vaya la luz. MEDIDAS DEL BOTE : - Sensor ubicado a 16.5 cm del fondo - 100% = agua a 12 cm del fondo -> distancia sensor-agua = 4.5 cm - 0% = agua en el fondo -> distancia sensor-agua = 16.5 cm - 1% = 0.12 cm de agua; volumen al 100% = ~1.14 L

```
 LIBRERIAS:
  - LiquidCrystal_I2C (Frank de Brabander)
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Preferences.h>

// ---------------------- CONFIGURACION DE PINES ----------------------
#define TRIG_PIN     5
#define ECHO_PIN     18

#define RELAY_EXTRAE  26   // IN1 - bomba que SACA agua
#define RELAY_LLENA   27   // IN2 - bomba que METE agua

#define BTN_MODO     32
#define BTN_SUBIR    33
#define BTN_BAJAR    25

#define LCD_SDA      21
#define LCD_SCL      22

#define RELAY_ACTIVO   LOW
#define RELAY_INACTIVO (!RELAY_ACTIVO)

// ---------------------- MEDIDAS DEL BOTE ------------------------------
const float ALTURA_SENSOR_CM     = 16.5;  // altura del sensor desde el fondo
const float ALTURA_AGUA_LLENO_CM = 12.0;  // altura del agua que es 100%
const float ALTURA_AGUA_VACIO_CM = 0.0;   // altura del agua que es 0%
const float DIAMETRO_BOTE_CM     = 11.0;

const float DIST_TANQUE_LLENO_CM  = ALTURA_SENSOR_CM - ALTURA_AGUA_LLENO_CM;  // 4.5 cm
const float DIST_TANQUE_VACIO_CM  = ALTURA_SENSOR_CM - ALTURA_AGUA_VACIO_CM;  // 16.5 cm
const float RADIO_BOTE_CM         = DIAMETRO_BOTE_CM / 2.0;
const float AREA_BASE_CM2         = PI * RADIO_BOTE_CM * RADIO_BOTE_CM;
const float TANQUE_VOLUMEN_LITROS = AREA_BASE_CM2 * (ALTURA_AGUA_LLENO_CM - ALTURA_AGUA_VACIO_CM) / 1000.0;
// ~1.14 L

// ---------------------- PARAMETROS DE CONTROL --------------------------
const int PASO_SETPOINT   = 1;    // cuanto cambia el MAX/MIN por pulsacion (%)
const int MARGEN_MINIMO   = 10;   // separacion minima entre MAX y MIN (%)
const unsigned long PAUSA_CAMBIO_BOMBA_MS = 1000;  // ambas apagadas al cambiar de bomba
const int MAX_ERRORES_SENSOR = 4; // lecturas fallidas seguidas antes de parar (~2 s)

// ---------------------- OBJETOS GLOBALES -------------------------------
LiquidCrystal_I2C lcd(0x27, 16, 2);  // si no se ve nada, prueba 0x3F
Preferences prefs;

// ---------------------- ESTADO DEL SISTEMA -----------------------------
enum EstadoCiclo { INICIANDO, LLENANDO, VACIANDO, ERROR_SENSOR };
EstadoCiclo estado = INICIANDO;

int nivelMax = 80;   // % - se carga de memoria en setup()
int nivelMin = 20;   // % - se carga de memoria en setup()

enum ModoEdicion { EDITAR_MAX, EDITAR_MIN };
ModoEdicion modoEdicion = EDITAR_MAX;

float nivelActualPorc   = 0.0;
float nivelActualLitros = 0.0;
int   erroresSensor     = 0;

unsigned long momentoCambioEstado = 0;

// Botones (deteccion de flanco + antirrebote)
```

```
bool anteriorModo  = HIGH;
bool anteriorSubir = HIGH;
bool anteriorBajar = HIGH;
unsigned long ultimoPulsoModo  = 0;
unsigned long ultimoPulsoSubir = 0;
unsigned long ultimoPulsoBajar = 0;
const unsigned long DEBOUNCE_MS = 200;

unsigned long ultimaLecturaSensor = 0;
const unsigned long INTERVALO_SENSOR_MS = 500;

unsigned long ultimaActualizacionLCD = 0;
const unsigned long INTERVALO_LCD_MS = 300;

// ---------------------- DECLARACION DE FUNCIONES ----------------------
float medirDistanciaCM();
void actualizarNivelAgua();
float mapFloat(float x, float in_min, float in_max, float out_min, float out_max);
void actualizarCiclo();
void cambiarEstado(EstadoCiclo nuevo);
void aplicarBombas();
void apagarBombas();
const char* nombreEstado();
bool botonPresionado(int pin, bool &anterior, unsigned long &ultimoPulso);
void manejarBotones();
void validarLimites();
void guardarConfiguracion();
void actualizarLCD();
void imprimirLinea(int fila, const char* texto);

// ========================================================================
void setup() {
  Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  pinMode(RELAY_EXTRAE, OUTPUT);
  pinMode(RELAY_LLENA, OUTPUT);
  apagarBombas();

  pinMode(BTN_MODO,  INPUT_PULLUP);
  pinMode(BTN_SUBIR, INPUT_PULLUP);
  pinMode(BTN_BAJAR, INPUT_PULLUP);

  // LCD: pequena espera para que arranque, e I2C a velocidad estandar
  delay(500);
  Wire.begin(LCD_SDA, LCD_SCL);
  Wire.setClock(100000);
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Iniciando...");

  // Cargar MAX y MIN guardados (si no hay nada guardado: 80 y 20)
  prefs.begin("config", false);
  nivelMax = prefs.getInt("nivelMax", 80);
  nivelMin = prefs.getInt("nivelMin", 20);
  prefs.end();
  validarLimites();

  Serial.println("=== Control de nivel - ciclo continuo ===");
  Serial.print("Distancia al 100%: "); Serial.print(DIST_TANQUE_LLENO_CM); Serial.println(" cm");
  Serial.print("Distancia al 0%:   "); Serial.print(DIST_TANQUE_VACIO_CM); Serial.println(" cm");
  Serial.print("Volumen al 100%:   "); Serial.print(TANQUE_VOLUMEN_LITROS, 3); Serial.println(" L");
```

```
  Serial.print("MAX: "); Serial.print(nivelMax); Serial.print("%  MIN: "); Serial.print(nivelMin); Serial.println("%");

  delay(1000);
  lcd.clear();
}

// ========================================================================
void loop() {
  manejarBotones();

  if (millis() - ultimaLecturaSensor >= INTERVALO_SENSOR_MS) {
    ultimaLecturaSensor = millis();
    actualizarNivelAgua();
    actualizarCiclo();
  }

  if (millis() - ultimaActualizacionLCD >= INTERVALO_LCD_MS) {
    ultimaActualizacionLCD = millis();
    actualizarLCD();
  }
}

// ========================================================================
// SENSOR: mide 5 veces y devuelve la mediana (filtra lecturas erroneas)
float medirDistanciaCM() {
  const int NUM_MUESTRAS = 5;
  float muestras[NUM_MUESTRAS];
  int validas = 0;

  for (int i = 0; i < NUM_MUESTRAS; i++) {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    long duracion = pulseIn(ECHO_PIN, HIGH, 30000);
    if (duracion > 0) {
      muestras[validas] = duracion * 0.0343 / 2.0;
      validas++;
    }
    delay(10);
  }

  if (validas == 0) return -1;

  for (int i = 0; i < validas - 1; i++) {
    for (int j = 0; j < validas - i - 1; j++) {
      if (muestras[j] > muestras[j + 1]) {
        float tmp = muestras[j];
        muestras[j] = muestras[j + 1];
        muestras[j + 1] = tmp;
      }
    }
  }
  return muestras[validas / 2];
}

void actualizarNivelAgua() {
  float distancia = medirDistanciaCM();

  if (distancia < 0) {
    erroresSensor++;
    Serial.println("Error leyendo el sensor ultrasonico");
    return;
  }
```

```
  erroresSensor = 0;

  float porcentaje = mapFloat(distancia, DIST_TANQUE_LLENO_CM, DIST_TANQUE_VACIO_CM, 100.0, 0.0);
  nivelActualPorc   = constrain(porcentaje, 0.0, 100.0);
  nivelActualLitros = (nivelActualPorc / 100.0) * TANQUE_VOLUMEN_LITROS;

  Serial.print("Distancia: ");   Serial.print(distancia, 1);
  Serial.print(" cm | Nivel: "); Serial.print(nivelActualPorc, 1);
  Serial.print(" % | ");        Serial.print(nivelActualLitros, 2);
  Serial.print(" L | Estado: "); Serial.println(nombreEstado());
}

float mapFloat(float x, float in_min, float in_max, float out_min, float out_max) {
  return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
}

// ========================================================================
// CICLO: LLENANDO hasta MAX -> VACIANDO hasta MIN -> LLENANDO ...
void actualizarCiclo() {
  // Seguridad: si el sensor falla varias veces seguidas, parar todo
  if (erroresSensor >= MAX_ERRORES_SENSOR) {
    if (estado != ERROR_SENSOR) cambiarEstado(ERROR_SENSOR);
    apagarBombas();
    return;
  }

  switch (estado) {
    case INICIANDO:
    case ERROR_SENSOR:
      // Primera lectura valida (o el sensor se recupero): decidir por donde empezar
      if (erroresSensor == 0) {
        cambiarEstado(nivelActualPorc >= nivelMax ? VACIANDO : LLENANDO);
      }
      break;

    case LLENANDO:
      if (nivelActualPorc >= nivelMax) cambiarEstado(VACIANDO);
      break;

    case VACIANDO:
      if (nivelActualPorc <= nivelMin) cambiarEstado(LLENANDO);
      break;
  }

  aplicarBombas();
}

void cambiarEstado(EstadoCiclo nuevo) {
  estado = nuevo;
  momentoCambioEstado = millis();
  apagarBombas();   // las dos apagadas durante la pausa de cambio
  Serial.print(">>> Cambio de estado: "); Serial.println(nombreEstado());
}

void aplicarBombas() {
  // Durante la pausa despues de un cambio, ambas bombas quedan apagadas
  if (millis() - momentoCambioEstado < PAUSA_CAMBIO_BOMBA_MS) {
    apagarBombas();
    return;
  }

  if (estado == LLENANDO) {
    digitalWrite(RELAY_EXTRAE, RELAY_INACTIVO);   // primero apagar la otra
    digitalWrite(RELAY_LLENA,  RELAY_ACTIVO);
  } else if (estado == VACIANDO) {
    digitalWrite(RELAY_LLENA,  RELAY_INACTIVO);
```

```
    digitalWrite(RELAY_EXTRAE, RELAY_ACTIVO);
  } else {
    apagarBombas();
  }
}

void apagarBombas() {
  digitalWrite(RELAY_EXTRAE, RELAY_INACTIVO);
  digitalWrite(RELAY_LLENA,  RELAY_INACTIVO);
}

const char* nombreEstado() {
  switch (estado) {
    case LLENANDO:    return "LLENANDO";
    case VACIANDO:    return "VACIANDO";
    case ERROR_SENSOR: return "ERROR SENSOR";
    default:          return "INICIANDO";
  }
}

// ========================================================================
// BOTONES
// Devuelve true solo en el instante en que el boton se presiona
bool botonPresionado(int pin, bool &anterior, unsigned long &ultimoPulso) {
  bool lectura = digitalRead(pin);
  bool presionado = false;
  if (lectura == LOW && anterior == HIGH && (millis() - ultimoPulso) > DEBOUNCE_MS) {
    ultimoPulso = millis();
    presionado = true;
  }
  anterior = lectura;
  return presionado;
}

void manejarBotones() {
  // MODO: alterna entre editar MAX y editar MIN
  if (botonPresionado(BTN_MODO, anteriorModo, ultimoPulsoModo)) {
    modoEdicion = (modoEdicion == EDITAR_MAX) ? EDITAR_MIN : EDITAR_MAX;
    Serial.print("Modo: "); Serial.println(modoEdicion == EDITAR_MAX ? "Max" : "Min");
  }

  // SUBIR
  if (botonPresionado(BTN_SUBIR, anteriorSubir, ultimoPulsoSubir)) {
    if (modoEdicion == EDITAR_MAX) {
      nivelMax += PASO_SETPOINT;
      if (nivelMax > 100) nivelMax = 100;
    } else {
      nivelMin += PASO_SETPOINT;
      if (nivelMin > nivelMax - MARGEN_MINIMO) nivelMin = nivelMax - MARGEN_MINIMO;
    }
    guardarConfiguracion();
  }

  // BAJAR
  if (botonPresionado(BTN_BAJAR, anteriorBajar, ultimoPulsoBajar)) {
    if (modoEdicion == EDITAR_MAX) {
      nivelMax -= PASO_SETPOINT;
      if (nivelMax < nivelMin + MARGEN_MINIMO) nivelMax = nivelMin + MARGEN_MINIMO;
    } else {
      nivelMin -= PASO_SETPOINT;
      if (nivelMin < 0) nivelMin = 0;
    }
    guardarConfiguracion();
  }
}
```

```
// Asegura valores validos (por si en memoria habia algo raro)
void validarLimites() {
  nivelMax = constrain(nivelMax, MARGEN_MINIMO, 100);
  nivelMin = constrain(nivelMin, 0, nivelMax - MARGEN_MINIMO);
}

void guardarConfiguracion() {
  prefs.begin("config", false);
  prefs.putInt("nivelMax", nivelMax);
  prefs.putInt("nivelMin", nivelMin);
  prefs.end();
  Serial.print("MAX: "); Serial.print(nivelMax); Serial.print("%  MIN: "); Serial.print(nivelMin); Serial.println("%");
}

// ========================================================================
// LCD 16x2
//   Fila 0: "Max:80% Min:20% "
//   Fila 1: "[Max]  75% 0.86L" (o "Error sensor" si el sensor falla)
void actualizarLCD() {
  char linea[17];

  snprintf(linea, sizeof(linea), "Max:%d%% Min:%d%%", nivelMax, nivelMin);
  imprimirLinea(0, linea);

  if (estado == ERROR_SENSOR) {
    imprimirLinea(1, "Error sensor");
  } else {
    // [Max] o [Min] = modo seleccionado, luego % actual y litros actuales
    // El % se alinea a 3 cifras para que el texto no "brinque" al cambiar
    snprintf(linea, sizeof(linea), "[%s] %3d%% %.2fL",
             modoEdicion == EDITAR_MAX ? "Max" : "Min",
             (int)(nivelActualPorc + 0.5),
             nivelActualLitros);
    imprimirLinea(1, linea);
  }
}

// Escribe un texto en la fila indicada y rellena con espacios hasta 16
// caracteres, para borrar restos de textos anteriores sin parpadeo
void imprimirLinea(int fila, const char* texto) {
  lcd.setCursor(0, fila);
  int n = 0;
  while (texto[n] != '\0' && n < 16) {
    lcd.print(texto[n]);
    n++;
  }
  while (n < 16) {
    lcd.print(' ');
    n++;
  }
}
```

Resultados

.

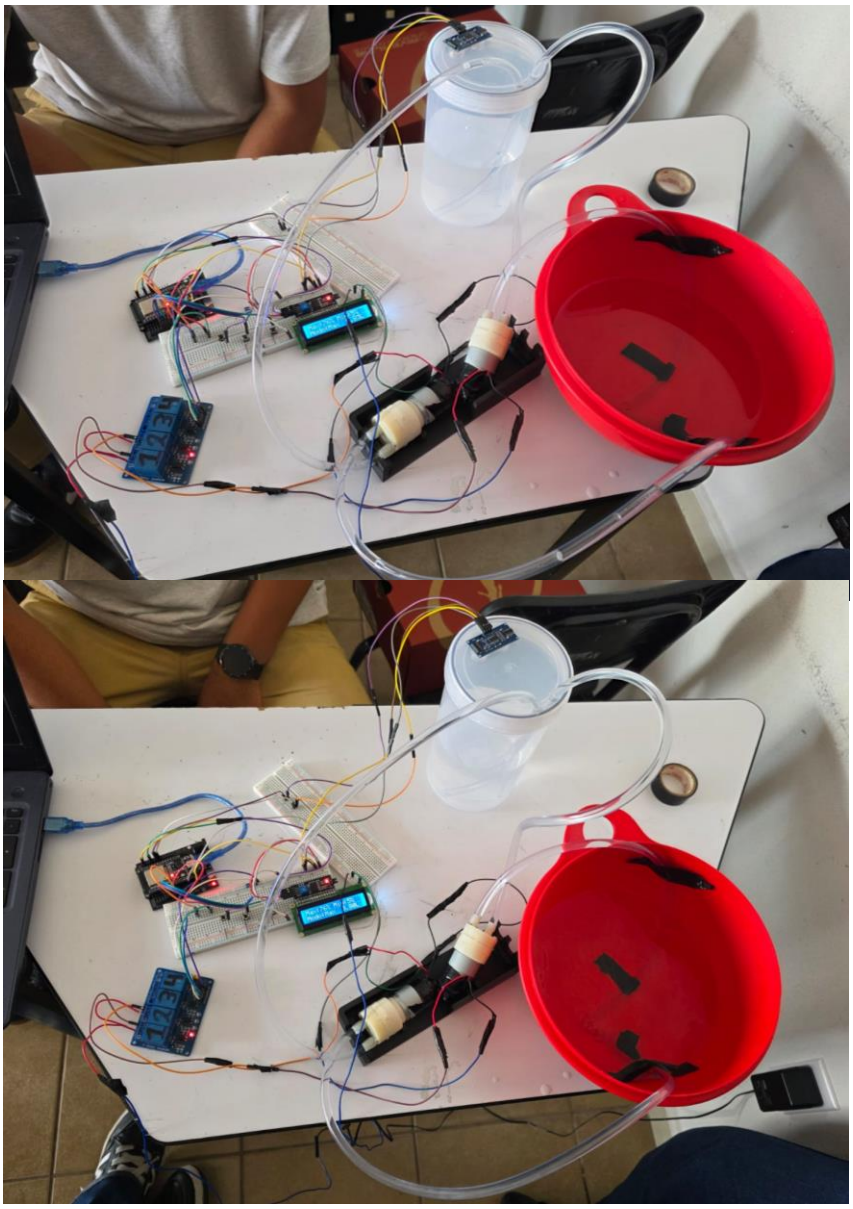


Figura 2. Montaje final del sistema de control de nivel de agua

Observaciones y Conclusiones

Observaciones

Durante la práctica se observó cómo el ESP32 puede controlar automáticamente el nivel de agua de un recipiente mediante un sensor ultrasónico. Las mediciones permiten conocer el porcentaje de llenado y calcular el volumen aproximado de agua en litros. También se comprobó la importancia de establecer niveles máximo y mínimo para controlar el funcionamiento de las bombas de llenado y vaciado.

Además, la pantalla LCD facilita la visualización de los datos del sistema, mientras que los botones permiten modificar los valores establecidos. La programación también contempla el almacenamiento de la configuración y la detección de errores del sensor.

Conclusiones

Con esta práctica se comprendió cómo se pueden integrar la programación y los componentes electrónicos para desarrollar un sistema automatizado de control del nivel de agua. Se aprendió la importancia de utilizar sensores para obtener mediciones, relevadores para controlar dispositivos y una pantalla para mostrar información en tiempo real.

En conclusión, el proyecto permite aplicar conocimientos de programación y electrónica en una situación práctica. Asimismo, ayuda a comprender cómo la automatización puede facilitar tareas de control y reducir la necesidad de intervención manual.

Referencias

[1] Espressif Systems. (s. f.). ESP32 Series Datasheet. Espressif Systems.

[2] Arduino. (s. f.). Arduino Documentation. Arduino.

[3] Mano, M. M., & Ciletti, M. D. (2018). Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog. Pearson.

[4] Bolton, W. (2021). Programmable Logic Controllers. Newnes/Elsevier.